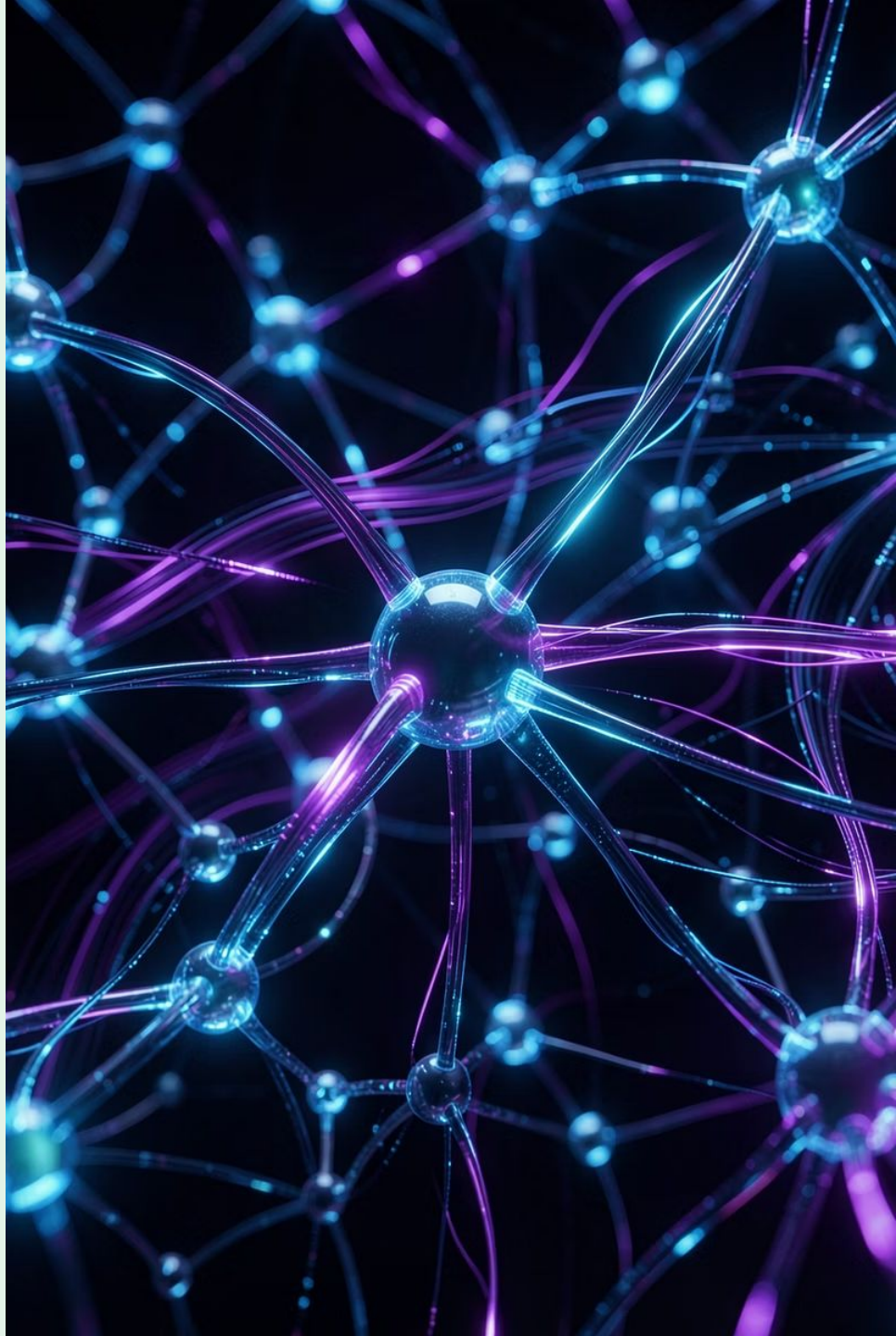
MESA AI-Native Track

Claude Basics

18 September 2026 · Cohort 4



Who am I?



Engg. from VNIT, Nagpur
Graduated 2013

Tata Motors
2013 – 2016

MBA from ISB
2016 – 2017

Startups – Growth & Customer Success
2017 – 2021

Founder, Furritas (Pet-care Startup)
2021 – 2023

Founder, Day One Consulting
2024 – 2026

Day One Consulting - Impact

12 Founders
Built their first AI-powered MVP

78 Professionals
Trained in Vibe-coding across functions

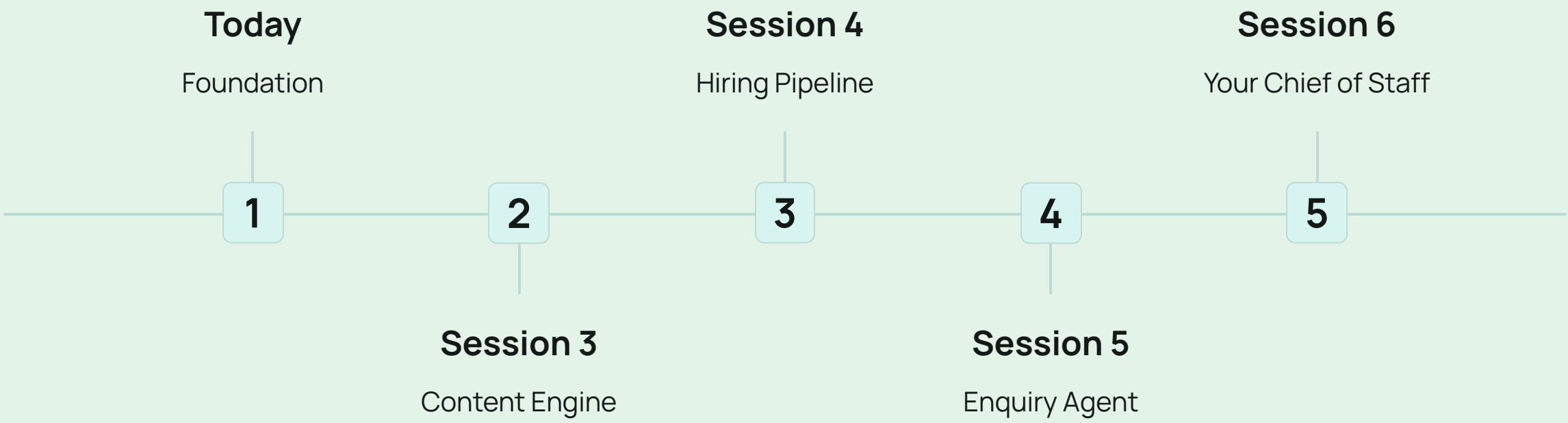
2 Companies, 5 Teams, 147 Members
AI Adoption Bootcamps at GEP & BSV

Demo Sessions
ISB AI Summit, TPF, The Builder's Club

900+ Students
'AI Fluency' course at KRM University

2 Hackathons Hosted
At KRM University and Insurity

The Month in 30 Seconds



Four real business problems. Four builds. One portfolio.

Large Language Models

What is a Large Language Model?

An AI system trained on immense datasets of text to understand and generate human language. They **predict** the most useful thing to say next, based on their training. They do not think or understand in the human sense.

The Frontier Models



GPT (OpenAI)

The model that brought LLMs mainstream. A strong general assistant, widely integrated.



Claude (Anthropic)

Known for strong reasoning, large context windows, and predictable instruction-following with a focus on AI safety.



Gemini (Google DeepMind)

Google's LLM family, integrated across its ecosystem. Flash variant excels at high-volume, mechanical tasks.



DeepSeek & Qwen (Open Source)

These models from DeepSeek AI and Alibaba demonstrate that frontier capability is no longer exclusive to major labs, offering competitive performance, especially in coding and multilingual tasks.



The landscape is rapidly evolving. The key is knowing how to evaluate new models as they emerge, rather than mastering every single one.

Why Claude for This Course?

Claude Code offers the most capable **agentic coding environment** available today, allowing it to see files, run commands, and iterate without manual copy-pasting.

Choosing a Model - The Decision Framework

Not every task deserves the same model. The right question is not **"which is best?"** but **"which is right for this job?"**

FIVE PARAMETERS THAT DRIVE THE DECISION

CONSEQUENCE OF BEING WRONG

Low: Gemini Flash, Haiku (wrong answer costs a retry)
High: Sonnet, Claude API (wrong answer costs trust)

VOLUME & COST

10 calls: use the most capable model
10,000 calls: multiply cost per call by 10,000 before deciding. (e.g., Gemini Flash processes 1,000 CVs for what Opus 5 charges for 20)

REASONING COMPLEXITY

Extracting a date: any model
Synthesizing documents: use a tier up
Writing defensible code: highest tier you can justify

LATENCY

Flash models: respond in under a second
Extended-thinking models: can take 30–60 seconds. (A voice agent making a caller wait 45 seconds is a failed agent.)

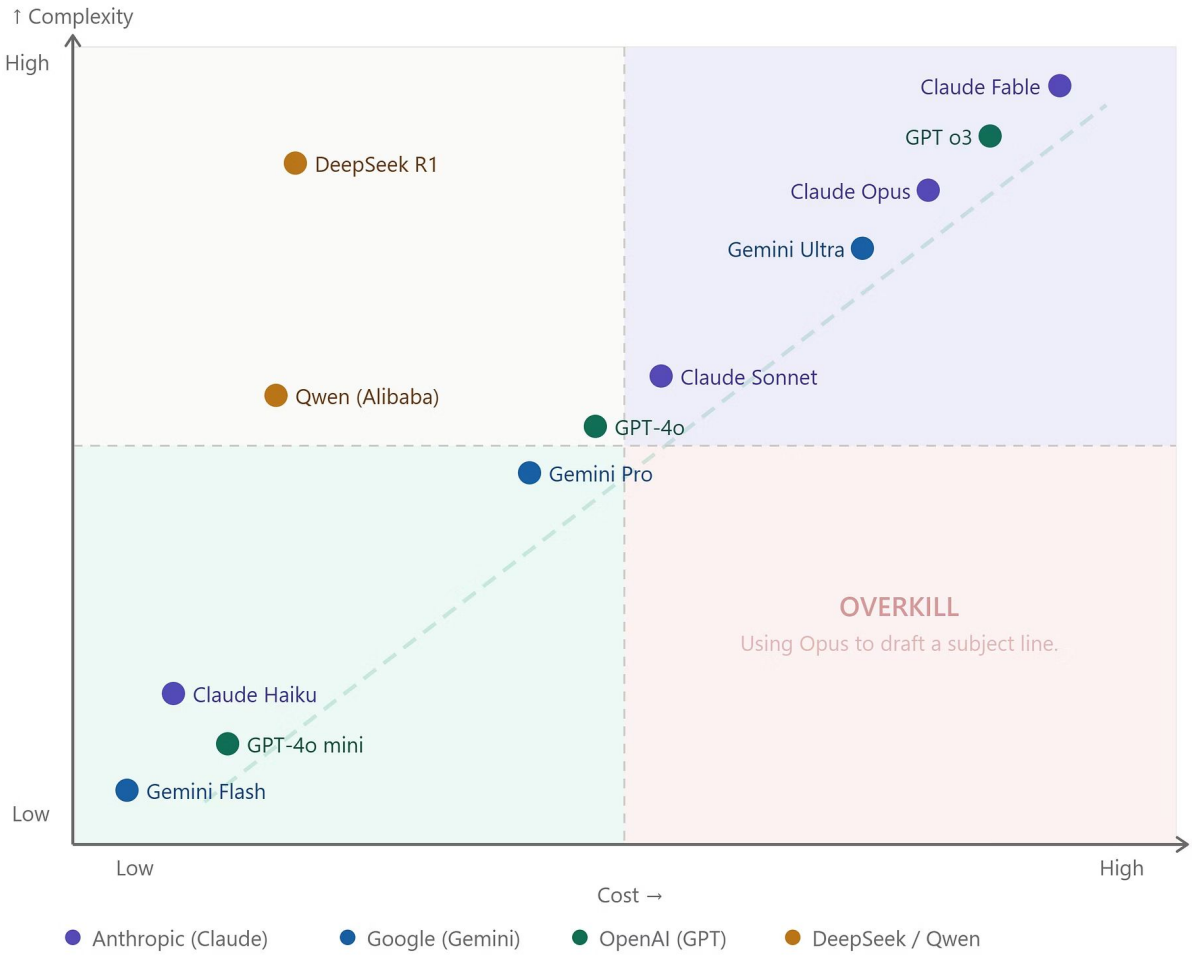
CONTEXT LENGTH NEEDED

Some tasks need the **whole document**.
Some only need **one paragraph**.
Sending more than you need costs money and slows everything down.

THE RULE OF THUMB

Start with the cheapest model that can do the job. Step up only when it visibly fails - not because a better model exists.

Which Model for Which Job?



The Cost Reality

Haiku, Low

Score 60 CVs → ‹₹1

Sonnet, Max

Same task → ~₹50

1,000×/day

Wrong effort = ₹6,000+/month

How Much is Too Much?

Context helps Claude work better — but there are real tradeoffs worth knowing.

What Context Gives You

- **Continuity** — Claude remembers what you built and decided.
- **Coherence** — Output improves with the full picture.
- **Speed** — No re-explaining every session.

Where it Works Against You

- **Token Cost** — Large contexts get expensive fast.
- **Quality Degradation** — Very long sessions get slower and worse.
- **Recency Bias** — Early instructions fade as the chat grows.
- **Hard Ceiling** — Oldest content drops off silently at the limit.
- **File Uploads** — Every attachment consumes context space.



Start a Fresh Chat When...

New task, major decision made, output quality drops, or after a milestone.



Carry Context Forward With...

An end-of-session summary, a CLAUDE.md file, or shared projects across chats.

Claude: Where You Do The Work

Two distinct modes, each optimized for different tasks. Know which one you're in before you start.

Chat & Cowork

Conversational — you type, Claude responds.

- Thinking through problems
- Drafting & researching
- Working with files & apps

Code

Agentic coding environment.

- Building apps & scripts
- Managing repositories
- Claude sees files, runs commands, iterates

Everything you **build** lives in the Code tab. Everything you **think through** lives in Chat.

Chats: Your Working Memory

A chat session retains all previous interactions within its context window for coherent, continuous assistance.

Use a Chat For:

- A single task or problem
- Research, drafting, or debugging
- Conversations where session continuity matters

Start a New Chat When:

- The task is complete or you hit a milestone
- Switching to an unrelated topic
- Output quality degrades from context overload
- You want a clean slate

One Chat ≠ One Project

Chats aren't for long-running project context — that's what **Projects** are for.

Pinned Chats

Pin chats you'll return to regularly. They stay visible in your sidebar for ongoing work or reference.

Projects: Your Persistent Context

Projects give Claude continuous, shared understanding across all related chats — not just organization, but persistent intelligence.

The Common Assumption

Projects are merely a folder system to keep chats tidy.

The Reality

Projects share a living context across every chat, so Claude always understands the bigger picture.

What This Means in Practice

- Every chat starts with your system prompt — instructions set once, applied always.
- Uploaded files are instantly accessible across all chats in the Project.
- Claude retains the Project's purpose and constraints, eliminating repetition.

When to Create a Project

- Work spanning multiple sessions needing consistent instructions.
- Client engagements, course materials, or product development.
- When Claude must always know certain things upfront.

When Not to Create a Project

- One-off tasks or quick questions with no follow-on work.
- Anything not requiring persistent or shared context.
- Random explorations that won't be revisited.

Decision Criteria: Does this work have its own rules, constraints, or recurring context? If yes - create a Project. If no - a Chat will suffice. *Example: This MESA AI-Native course is a Project - every chat automatically knows who I am, the curriculum, and how to respond.*

Artifacts: Your Productive Output

When Claude produces something you'll use outside the conversation, it creates an **Artifact** — a tangible output designed for action.

What Counts as an Artifact

- **Code** — scripts, components, applications
- **Documents** — reports, proposals, write-ups
- **Data** — tables, spreadsheets, JSON objects
- **Images** — visuals and diagrams

Where Artifacts Live

Stored in the chat and your **Artifacts library** — accessible from the sidebar anytime.

Edit & Iterate

Modify and refine generated content.

Download & Share

Export for other tools or teammates.

Versioned & Reusable

Build on previous iterations easily.

Habit to Build: Expect an Artifact — if you get plain text, ask Claude to "put the output in an Artifact."

Scheduled: Claude That Works While You're Not There

Set tasks for Claude to run automatically at a specific time or recurring cadence — continuous productivity even when you're offline.

What You Can Schedule

Daily/Weekly Reports

"Every Monday, summarise my open items."

Recurring Research

"Every Friday, pull the latest news on [topic]."

Contextual Reminders

"On the 1st of each month, draft my client update."

Monitoring Tasks

"Alert me if [condition] is met."

Chat vs. Scheduled

- A chat requires you to be present
- A Scheduled task runs whether you're there or not.

When to Use It

- **Regular Cadence**: repeating tasks that start the same way each time
- **Autonomous Work**: no input needed to begin, only review at the end
- **Fixed Deliverables**: recurring outputs with a consistent format

Note: This is a standing instruction to Claude - with a timer - not technical automation. Example: Every Sunday, "Summarise the week's conversations." You read it Monday morning.

CLAUDE.design: From Idea to Visual

A dedicated canvas for visual work — beyond chat or code — offering a true design surface for creative output.

What You Can Build

Landing Pages & One-Pagers

Mock up marketing assets or simple websites.

Pitch Decks & Presentations

Craft compelling visual narratives.

Mockups & UI Prototypes

Visualize interfaces and app flows.

Flyers, Social Posts, Proposals

Generate diverse visual materials.

Artifact vs. Design

Artifact: A file exported for use elsewhere (Word, JSON, code snippet).

Design: A canvas you iterate on visually — edit, publish, share — within Claude.

When to Use Design

- The output **IS** the visual
- You'll share or publish it directly
- You need visual review before finalising

When Not to Use

- Building a live app → use Code
- Editing in Word/Google Docs → use Artifact
- Internal reference only → chat Artifact is faster

Example: This deck was built in Gamma with content from Claude — reasoning in Claude, designing in a visual tool — the ideal workflow.

Customize: Extend What Claude Can Do

Not about how Claude writes. About what Claude can access and how it behaves across your work. Three tabs. Three different types of extension.

Skills

Reusable instruction sets for recurring types of work. Created by you, or installed from Anthropic and partners. Yours tab shows what you've built. Discover tab shows what's available.

Connectors

Permissions that let Claude access external apps on your behalf — Google Drive, Gmail, Slack, Calendar. Claude can read, write, and act inside those tools without you copy-pasting anything.

Plugins

Bundles of Skills, Connectors, and commands packaged together for specific workflows or teams. A complete setup in a single install.

1

Skills

What Claude knows how to do

2

Connectors

What Claude has access to

3

Plugins

Pre-built combinations of both

CLAUDE.md

A file in your project folder. Claude reads it automatically at the start of every session.

```
# Trip to Bali

## THE TRIP
Travelling with my partner.
Dates: 20 Dec – 3 Jan.
Budget: Mid-range. Not backpacker, not luxury.
We've been to Bali before — skip the obvious tourist stops.

## WHAT WE CARE ABOUT
- Boutique stays, not resorts — local feel preferred
- Mix of culture, food, and some beach downtime
- Minimal planning overhead — keep suggestions simple
```

Every new chat in this project starts here - Claude already knows the context before you type a word.

Same Task. Different Context.

Without CLAUDE.md

You type: `"Find me a villa in Seminyak for 27 Dec to 3 Jan"`

Claude replies: "Sure! How many guests? What's your budget? Do you prefer a private pool? Which area of Seminyak? Are you looking for something closer to the beach or the restaurants?"

With CLAUDE.md

You type: `"Find me a villa in Seminyak for 27 Dec to 3 Jan"`

Claude replies: "Here are three mid-range private villas in Seminyak for 27 Dec – 3 Jan, walkable to the restaurant strips. Filtered out resorts and anything with group facilities. All have direct booking options — no tour packages."

> Same prompt. Same model. Same question. The difference is not how you asked - it's what Claude already knew.

Claude Skills: Reusable Instructions for Recurring Work

A Skill is something you teach Claude once and never have to explain again.

Without a Skill

Every time you ask Claude to review your resume, you type the same thing:

> "Keep it to one page. Use action verbs.
> Don't use the word 'responsible for'."

Every. Single. Time.

With a Skill

You create a Skill called: `Review My Resume`

Claude knows all of that permanently — next time, just say: `"Review this."`

Skill vs. CLAUDE.md

- CLAUDE.md is for a project
- A Skill is for a type of work, across any chat

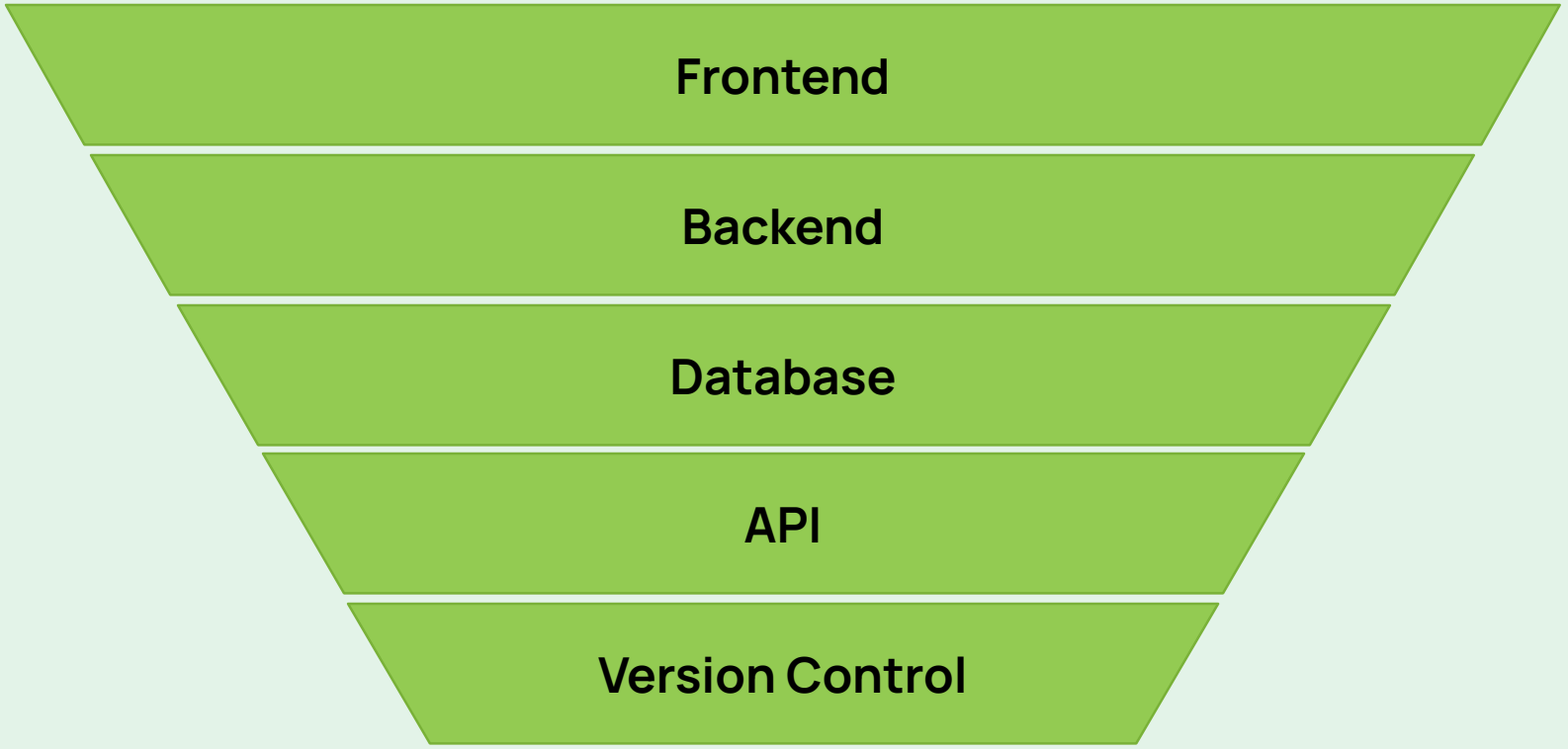
Examples

- *"Write in my tone"* — your voice, saved once
- *"Summarise meeting notes"* — your format, every time
- *"Review my code"* — your standards, not defaults

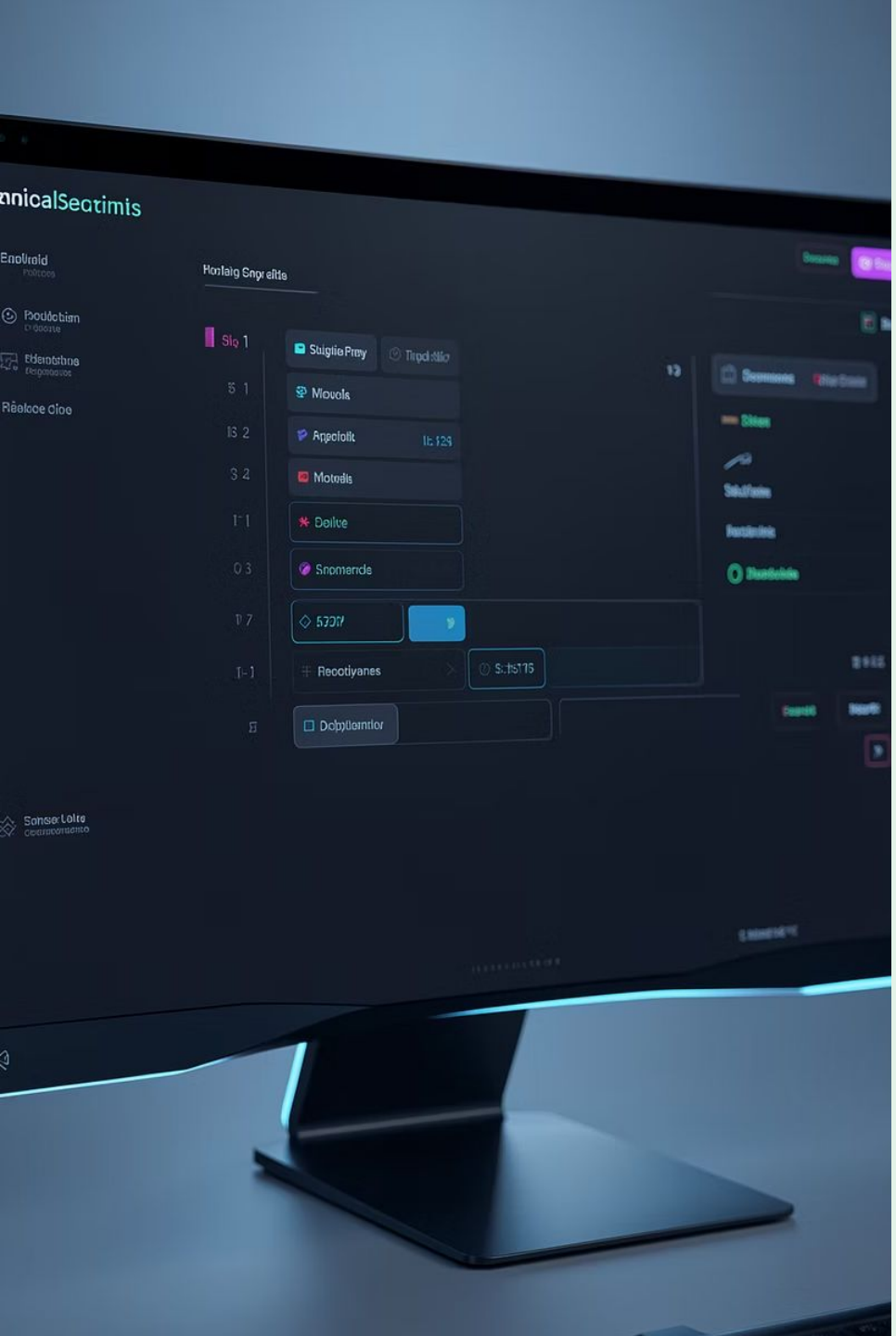
> In Case 1, we turn a founder's writing voice into a Skill — used automatically across every post.

The Five Parts

An app typically has five main components.



As Claude builds today, your only job is to point at the code and name the layer it belongs to.



Frontend

What the user sees and touches. Buttons, forms, text on a page.

If you can see it and click it — it's frontend.



Backend

The **logic** that runs when something happens. Users never see it — they feel it when it's slow.

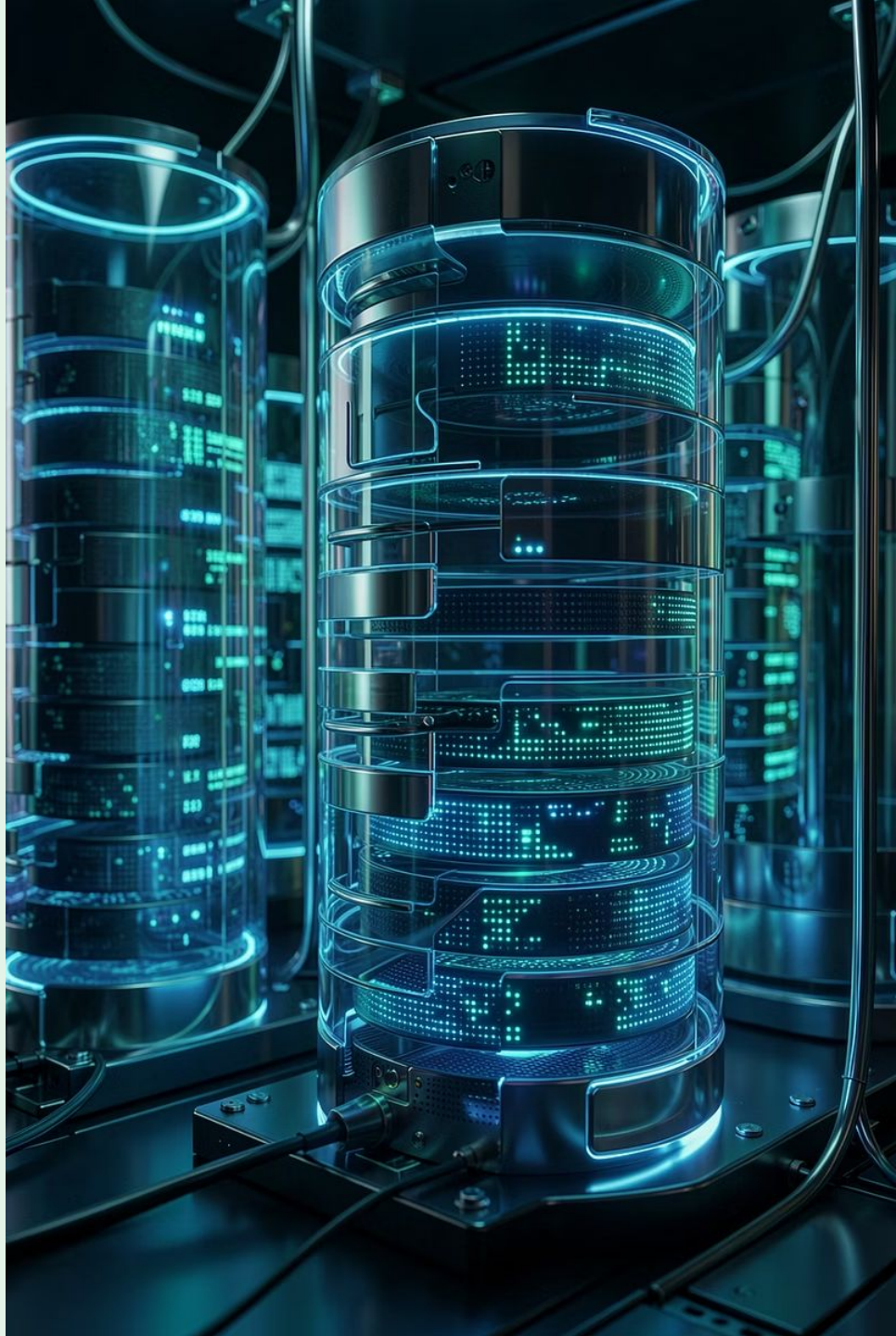
- User clicks submit → backend decides what happens next.

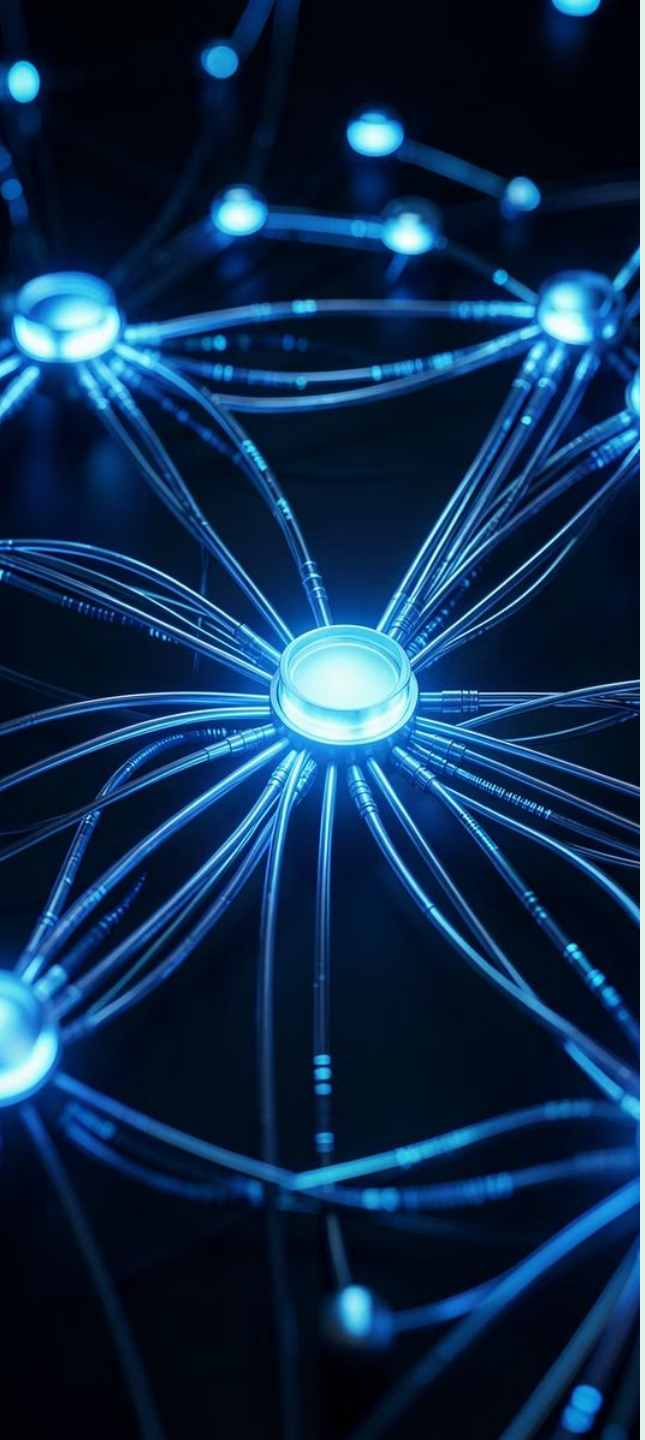
Database

Where things are stored so they survive a page refresh.

Without a database - everything disappears when the tab closes.

Supabase is what we'll use today.





API: How Your App Talks to the World

The Waiter Analogy

You don't walk into the kitchen and cook your own food. You tell the waiter what you want — they get it and bring it back.

1. **Your App**
   The customer
2. **API**
   The waiter
3. **The Service**
   The kitchen

> "Your app never goes to the kitchen. It always uses the waiter."

In Practice

- App wants AI to write something → calls Claude's API → gets response back
- App wants to send a message → calls Telegram's API → "Send this."

Every service this month — Claude, Gemini, Supabase, Telegram — has an API.

Same Idea - Levels of Complexity

APIs allow apps to talk to each other, but the complexity of these conversations can vary greatly.



Simple

Your weather app asks a weather API for conditions at a location. **One waiter, one kitchen.**



Medium

A contact form submission saves data (API 1) and sends a confirmation email (API 2). **Two waiters, two kitchens.**



Complex

A sales rep closes a deal in Salesforce. Automatically: an invoice is generated in QuickBooks (API 1), a welcome email goes to the client (API 2), and a folder is created for them in Google Drive (API 3). One trigger, three systems updated.



Agentic

A voice call comes into a company's support line. The call is transcribed in real time (API 1), Claude reads the transcript and drafts a resolution (API 2), the resolution is logged in the CRM (API 3), and the customer gets a follow-up email (API 4).

Every app you build this month orchestrates multiple APIs. Your code is the manager - telling the waiters what to fetch and in what order.

API Keys: Your App's Passwords

When your app wants to use Claude, Supabase, or Telegram, those services need to know: who is asking? The answer is an API key — a unique password that identifies your app and grants it access.

Think of it Like a Hotel Key Card

- The hotel doesn't let anyone walk into any room
- You get a key card at check-in — it only works for your room
- If you lose it, report it and get a new one immediately

Your API key works the same way. If someone else gets it — they walk in as you, use your account, and you get the bill.

> Lose one carelessly — someone else pays for their work using your name and your account. We'll make sure it doesn't happen to you.

The .env File — Where Keys Live

Not in your code. Not in a message. Not pasted into a prompt. In one place — a file called `.env`

Think of it Like a Hotel Safe

- Everything valuable goes in the safe
- Not visible to anyone walking past
- Never packed into your suitcase — it stays in the room

Your .env file is that safe. It holds all your keys and never leaves your computer — ever.

How It Works

```
CLAUDE_API_KEY=sk-ant-...
SUPABASE_KEY=eyJ...
TELEGRAM_TOKEN=7391...
```

Your .gitignore file tells GitHub: "Don't touch this. Don't upload this. Don't show this to anyone." — the .env file is always listed there.

> "Claude Code will remind you to set this up. Always say yes. If it doesn't — you remind it."

Three Rules. No Exceptions.

You don't need to understand everything about security today. You need these three habits - starting now.

1 Keys go in .env

Not in your code. Not in a chat. Not in a shared doc. In `.env` only.

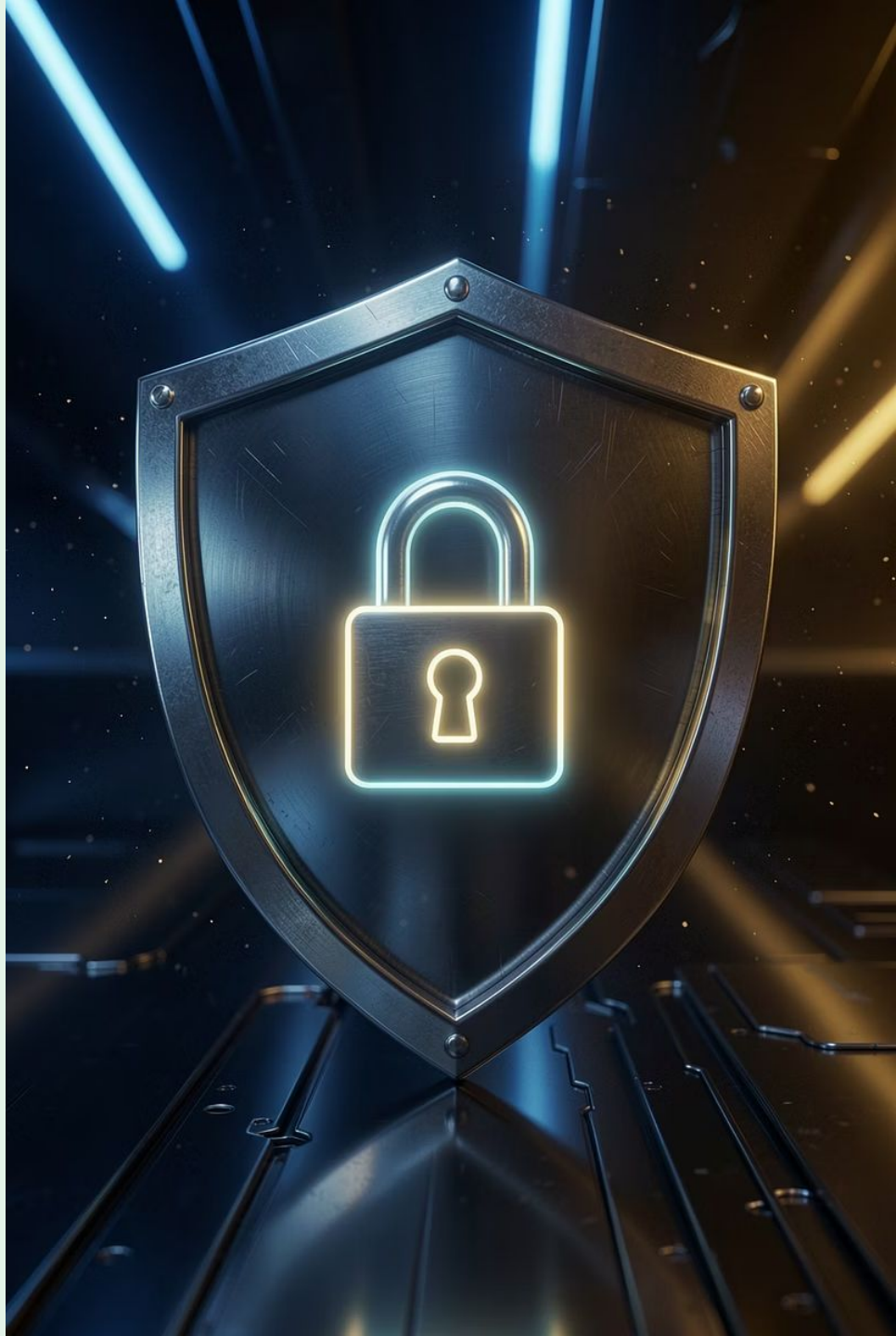
2 .env goes in .gitignore

Before your first GitHub push. If it's not listed, your keys are visible to the entire internet the moment you push.

3 If a key leaks - rotate it immediately

Not tomorrow. Every service lets you delete and regenerate a key in under a minute.

> "A developer pushed code with an AWS key visible. Four hours later — the bill was $50,000. These three rules exist so that never happens to you."



Version Control: Undo Button With a Memory

Without Version Control

```
Report_v1.docx
Report_FINAL.docx
Report_FINAL_actualfinal.docx
Report_FINAL_use_this_one.docx
```

Everyone has done this. It's chaos.

With GitHub

Every meaningful change gets one line:

```
"Added database connection"
"Fixed the login bug"
"Built the triage function"
```

- Full history stored automatically
- Go back to any point
- Work with others without overwriting

What We're Building Today

📋 App Description

I am building **a movie and TV show matchmaker** for **two people who can never agree on what to watch tonight**, so that **they can stop scrolling, stop negotiating, and actually watch a movie/TV show that both of them like - with exactly where to watch it in India, right now**.

Preparation for Launch

Before we dive into the build, we'll establish the foundational elements and review our core instructions.



GitHub Configuration

Setting up our repository for version control and collaborative development.



Supabase Backend

Initializing our data storage, authentication, and real-time capabilities.



The Master Prompt

Understanding the precise instructions that will guide Claude's code generation.



External API Access

Connecting our app to Claude and other services to power its features.

The Prompt

📝 **Build Prompt**

> A wrong plan takes 10 seconds to fix. Wrong code takes 20 minutes to fix.

Build

Live build - approximately 20 minutes

Frontend

UI components appearing

Backend

Functions and logic Claude writes

Database

Supabase wired in

API

Calls to Claude or other services

Version Control

Structure taking shape in files

Name the layer when you see it appear.

The Nine Checks - Does This Deserve to Be Built?

🛑 Kill Switches

Any No means: don't build it.

- 01 Problem real? YES — two people, one screen, twenty minutes of scrolling.
- 02 Workflow repeated? YES — every movie night. Same friction.
- 03 Input available? YES — preferences, TMDB data, live OTT availability.

⚖️ Sizing

Any No means: build something smaller.

- 04 Output valuable? YES — a specific title, on a platform they have, tonight.
- 05 Impact measurable? YES — time saved, post-watch ratings, quality over time.
- 08 ROI worth it? YES — hours to build. Negligible cost. Used indefinitely.

🔒 Boundary

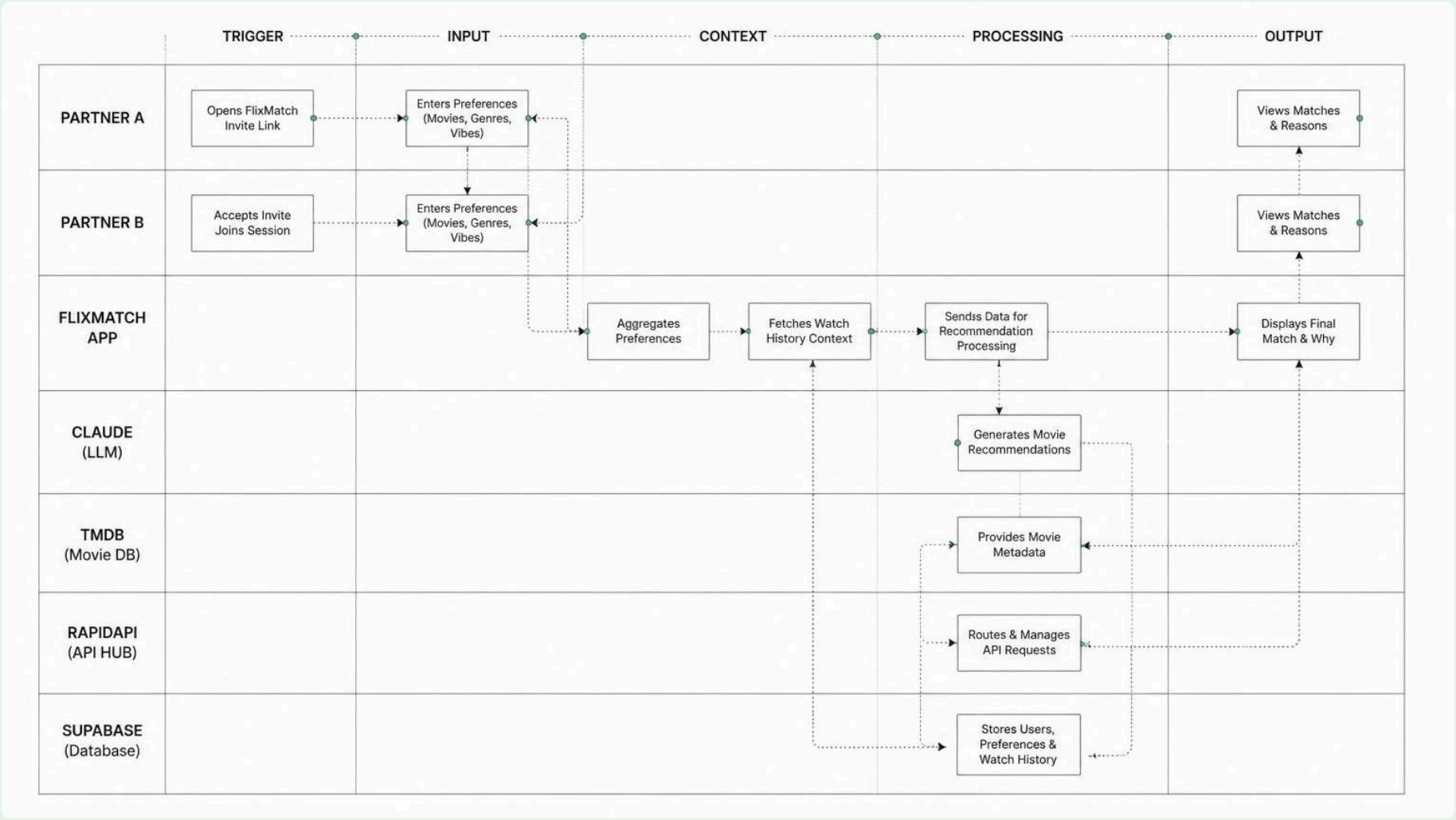
Any No means: put a human exactly here.

- 06 Failure risk OK? YES — worst case is a bad recommendation. Free to ignore.
- 07 Judgment protected? YES — neither partner sees the other's swipes.
- 09 Owner clear? YES — inputs owned individually. Final call belongs to both.

"Just decide for us — skip the swiping." → Killed by 07. The algorithm narrows. The humans choose.

"Show us each other's swipes in real time." → Killed by 07. That's the negotiation we came here to avoid.

FlixMatch — Components Map



Pushing to GitHub - Claude Code Does This For You

You don't need to know a single Git command.

How to Push

Open Claude Code and type:

> Connect this project to GitHub and push my code.

Claude Code will:

- Create the repository
- Connect your project
- Push everything up

The Push Habit

Every session ends with one instruction to Claude Code:

> Push my latest changes to GitHub.

- Not a submission — a checkpoint
- Saturday's build starts from your last push
- No push? Saturday starts from nothing

> One instruction. Thirty seconds. Before you leave today — tell Claude Code to push.

What You Built Today

All five parts. First session.

Frontend

App UI

Backend

Claude Code logic

Database

Supabase

API

Claude API

Version Control

GitHub

Before Monday

What to do before Session 2 on 21 September:

- [ ] Create a Vercel account tonight — link in the group
- [ ] Keep your laptop charged and build accessible
- [ ] Thursday opens with deploying what you built today
- [ ] Prepare for the assessment - everything you've learnt so far

---

Session 2 opens with Vercel. We deploy what you built today, get a live URL, and then the assessment begins.

You're walking in with something already built.